Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №3
по дисциплине «Устройство и функционирование интеллектуальных информационных систем»
«Введение в обработку текста (NLP Basics) на русском языке»

ЛР 09.02.08.02.003

Студент группы 1ИИС-41 Н.М. Аникин
группа и.о., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень и.о., фамилия

Барнаул 2026

Лабораторная работа №3

Введение в обработку текста (NLP Basics) на русском языке

Цель работы: изучить базовые этапы предобработки текста: токенизацию, очистку с помощью регулярных выражений, удаление стопслов, лемматизацию с помощью spaCy и NLTK, а также векторизацию текста методом TF-IDF.

Задачи к работе:

1. Установить и загрузить необходимые библиотеки (spaCy, русскоязычную модель ru_core_news_sm, NLTK со списком стоп-слов).

2. Сформировать исходный корпус текстов на русском языке.

3. Выполнить очистку текста от «шума» с помощью регулярных выражений (RegEx).

4. Провести токенизацию, удаление стоп-слов и лемматизацию с использованием spaCy.

5. Обернуть все этапы предобработки в единый пайплайн и обработать весь корпус.

6. Вручную рассчитать метрику TF-IDF для векторизации документов.

7. Выполнить самостоятельные задания по кастомизации стоп-слов, исследованию масштабируемости корпуса и модификации регулярных выражений.

Задание к работе: выполнить задачи 1–7 в соответствии с условиями. Ответить на кнтрольные вопросы.

Задание принял:  Аникин Н.М.

Подпись

Ход работы

Ссылка на GitHub-репозиторий: https://github.com/itsqwit/anikin_1iis41

Ссылка на блокнот:
https://colab.research.google.com/drive/1bz6Xz0gguWQo69dhmCpR2KIkdtWayBTB?usp=sharing

Шаг 1: Установка и загрузка библиотек

```
!pip install spacy
!python -m spacy download ru_core_news_sm

Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (0.27.3)
Requirement already satisfied: click<9.0.0,>=8.2.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.5.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.1.3)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.14.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/dist-packages (from spacy) (80.10.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (26.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (0.8.0)
Requirement already satisfied: pydantic-core==2.50.1 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (2.50.1)
Requirement already satisfied: typing-extensions>=4.16.0 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.4 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (0.4.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.7.22)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from thinc<8.4.0,>=8.3.12->spacy) (1.3.3)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (0.0.5)
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.13/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.26.0)
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.13/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (8.0.4)
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python3.13/dist-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.28.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->spacy) (3.0.4)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (4.15.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (0.16.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13.8.0->typer<1.0.0,>=0.3.0->spacy) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13.8.0->typer<1.0.0,>=0.3.0->spacy) (2.21.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart-open>=5.2.1->weasel<2.0.0,>=1.0.0->spacy) (2.5.1)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer<1.0.0,>=0.3.0->spacy) (0.1.2)
Collecting ru-core-news-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/ru_core_news_sm-3.8.0/ru_core_news_sm-3.8.0-py3-none-any.whl (15.3 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 15.3/15.3 MB 78.0 MB/s eta 0:00:00
Collecting pymorphy3>=1.0.0 (from ru-core-news-sm==3.8.0)
  Downloading pymorphy3-2.0.6-py3-none-any.whl.metadata (2.4 kB)
Collecting dawg2-python>=0.8.0 (from pymorphy3>=1.0.0->ru-core-news-sm==3.8.0)
  Downloading dawg2_python-0.9.0-py3-none-any.whl.metadata (7.5 kB)
Collecting pymorphy3-dicts-ru (from pymorphy3>=1.0.0->ru-core-news-sm==3.8.0)
  Downloading pymorphy3_dicts_ru-2.4.417150.4580142-py2.py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: setuptools>=68.2.2 in /usr/local/lib/python3.13/dist-packages (from pymorphy3>=1.0.0->ru-core-news-sm==3.8.0) (80.10.2)
Downloading pymorphy3-2.0.6-py3-none-any.whl (53 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53.9/53.9 kB 2.1 MB/s eta 0:00:00
Downloading dawg2_python-0.9.0-py3-none-any.whl (9.3 kB)
Downloading pymorphy3_dicts_ru-2.4.417150.4580142-py2.py3-none-any.whl (8.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.4/8.4 MB 69.7 MB/s eta 0:00:00
Installing collected packages: pymorphy3-dicts-ru, dawg2-python, pymorphy3, ru-core-news-sm
Successfully installed dawg2-python-0.9.0 pymorphy3-2.0.6 pymorphy3-dicts-ru-2.4.417150.4580142 ru-core-news-sm-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('ru_core_news_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

Рисунок 1 – установка необходимых библиотек

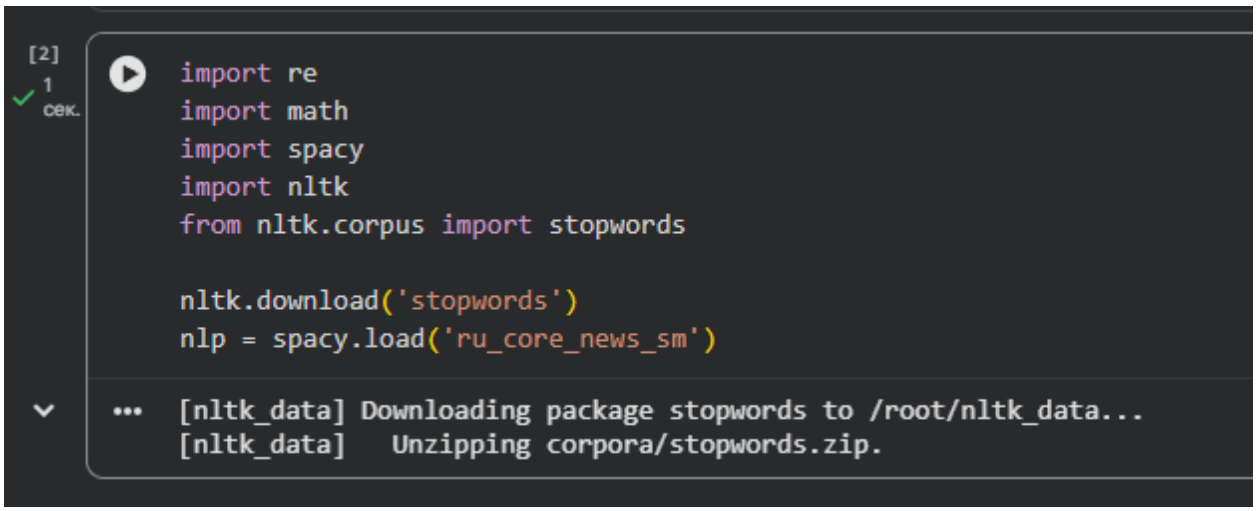


Рисунок 2 – загрузка библиотек

Шаг 2: Исходные данные (Корпус текстов)

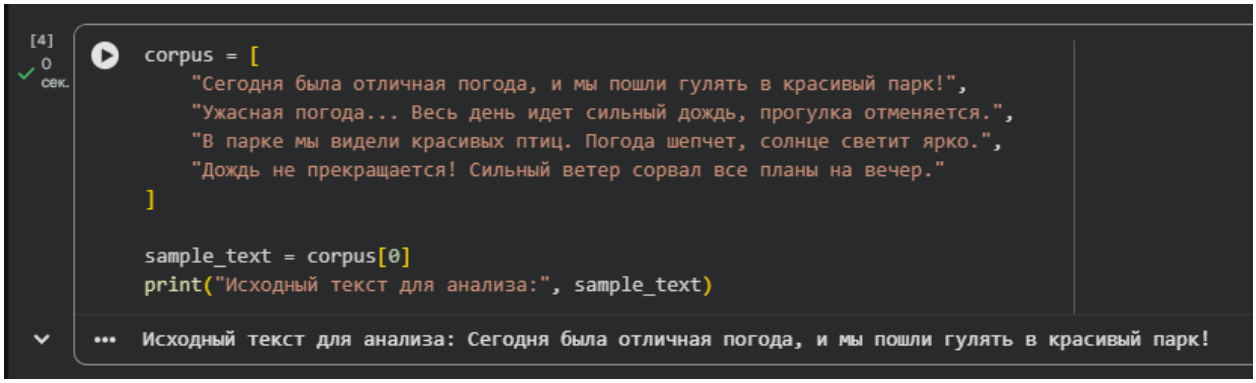


Рисунок 3 – определение корпуса из четырёх фраз

Шаг 3: Очистка текста с помощью Регулярных выражений (RegEx)

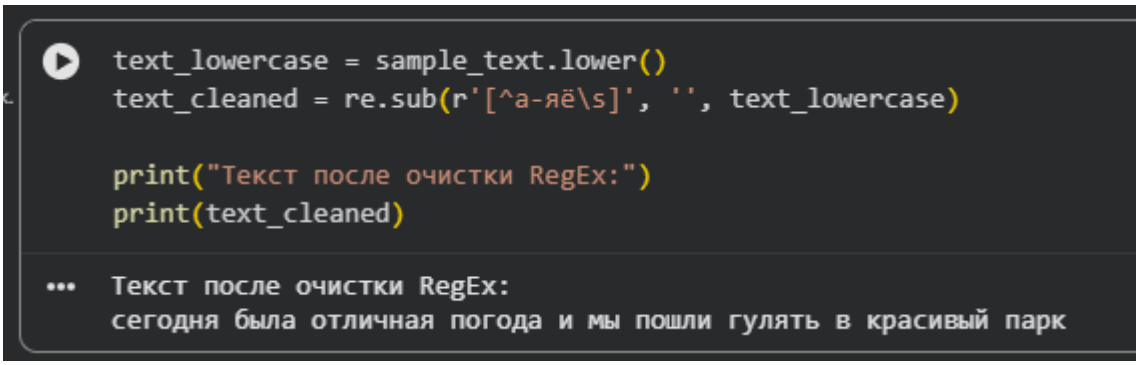


Рисунок 4 – очистка текста

Шаг 4: Токенизация, удаление стоп-слов и Лемматизация (spaCy)

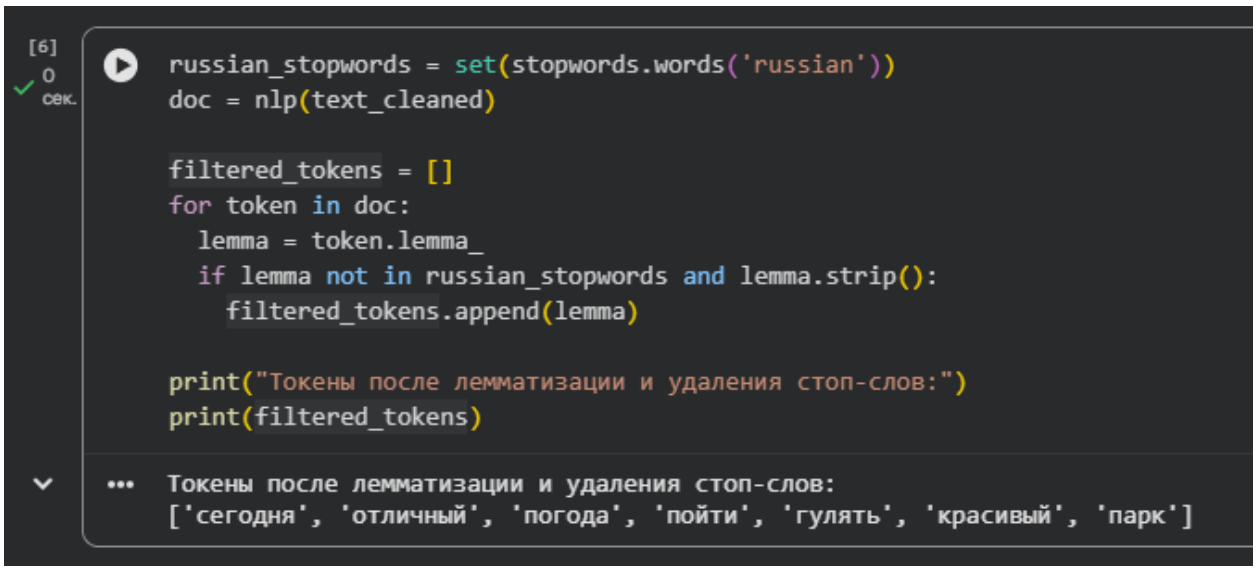


Рисунок 5 – токенизация, фильтрация и лемматизация очищенного текста

Шаг 5: Полная предобработка всего корпуса

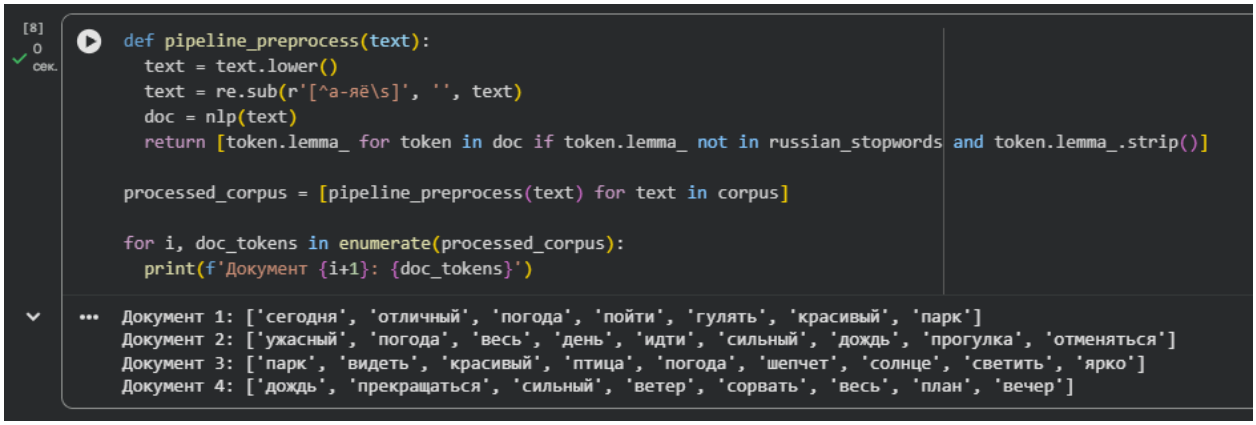


Рисунок 6 – предобработка всего корпуса с помощью функции

Шаг 6: Векторизация текста (Расчет TF-IDF вручную)

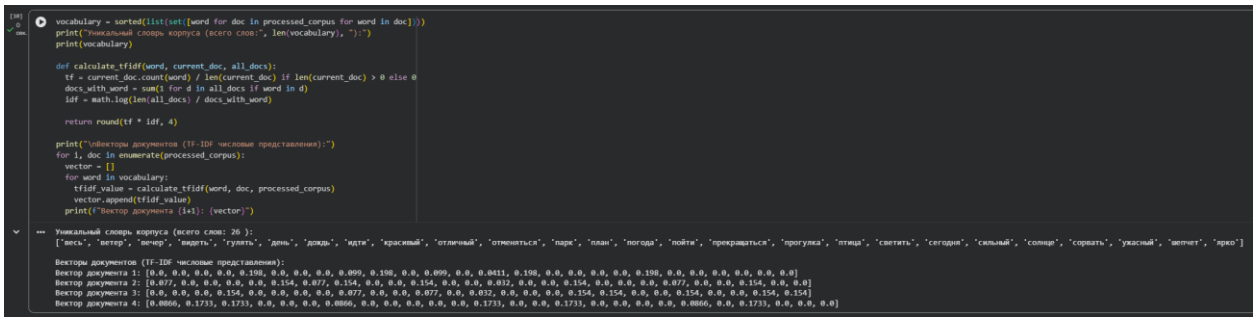


Рисунок 7 – расчёт TF-IDF

Задания для самостоятельной работы

Задание 1. Глубокий анализ и кастомизация стоп-слов

1. Напишите код, который выводит на экран общее количество слов в загруженном списке russian_stopwords, а также сам список (выведите его в алфавитном порядке).

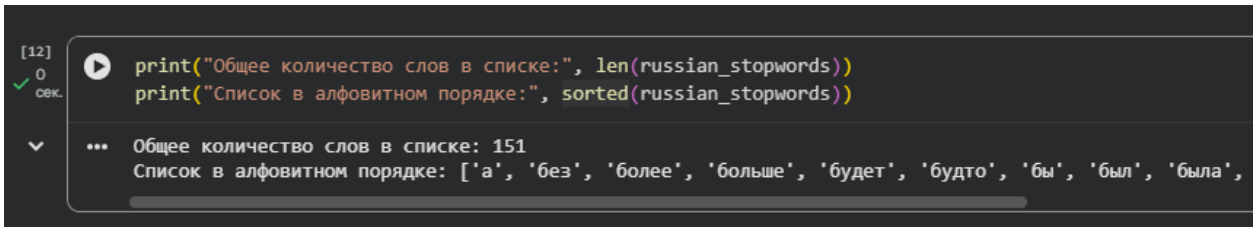


Рисунок 8 – вывод количества слов и отсортированного списка russian_stopwords

2. Найдите, выпишите в отчет и классифицируйте 5 любых стоп-слов (например: 1 личное местоимение, 1 предлог, 1 союз, 1 частица, 1 указательное слово).

№	Слово	Часть речи	Лингвистическая роль
1	я	Личное местоимение	Указывает на говорящее лицо (1-е лицо, единственное число), замещает имя существительное и выступает в роли подлежащего
2	для	Предлог	Выражает зависимость существительного от других слов, указывает на цель, назначение или пользу действия/предмета.
3	но	Союз (противительный)	Связывает однородные члены предложения или части сложного предложения, выражая отношения провипоставления или сопоставления.
4	бы	Частица (формообразующая)	Служит для образования формы условного наклонения глагола, выражает возможное, желдаемое или предполагаемое действие.
5	эти	Указательное местоимение	Служит для выделения конкретных объектов во множественном числе среди других, указывает на близкие во времени или пространстве предметы.

Таблица 1 – классификация пяти стоп-слов

3. Добавьте в список стоп-слов два кастомных контекстно-зависимых слова, которые часто встречаются в нашем корпусе (например, слова «сегодня» и «весь»). Очистите корпус заново и покажите в отчете, как изменился итоговый список токенов для первого документа.

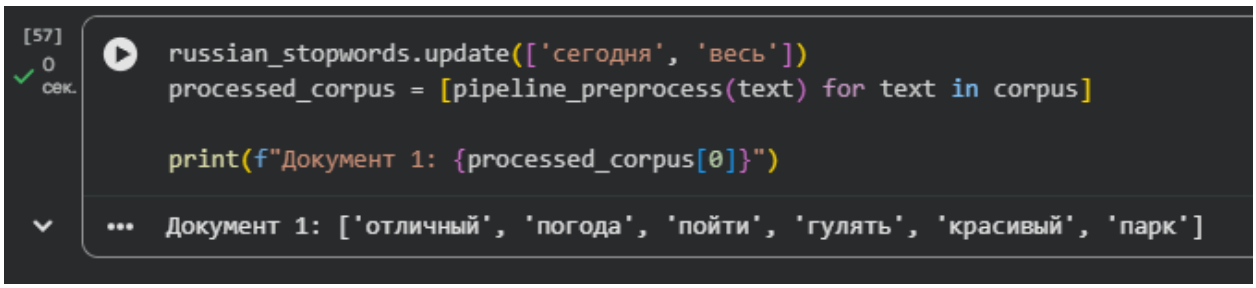


Рисунок 9 – итоговый список токенов для первого документа

Я добавил слова «сегодня» и «весь» в список стоп-слов. После повторной очистки корпуса из списка токенов для первого документа исчезло слово «сегодня».

Задание 2. Исследование масштабируемости корпуса и динамики векторов

1. Придумайте и добавьте в список corpus пятый документ на русском языке. Обязательное условие: предложение должно содержать как минимум два слова, которых ранее не было в словаре, и два слова, которые уже встречались в прошлых документах.

Новый документ: «Сейчас я куплю новый зонт, потому что на улице идёт сильный дождь.»

Новые слова: купить, новый, зонт, улица. Старые слова: идти, сильный, дождь.

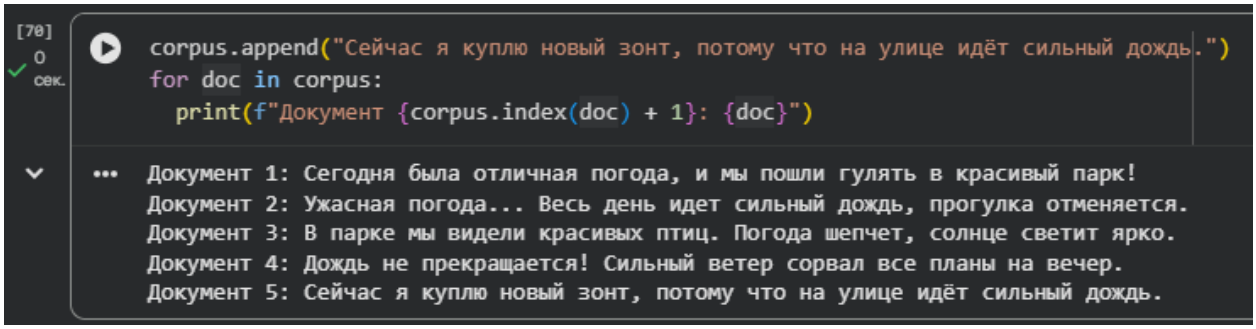


Рисунок 10 – в список добавлен пятый документ

2. Перезапустите пайплайн обработки. Сравните и зафиксируйте в отчете:

- Каким был размер уникального словаря (vocabulary) до добавления текста, и каким он стал после?
- Как изменилась длина итоговых числовых векторов для каждого документа?



Рисунок 11 – до добавления новых стоп-слов и пятого документа



Рисунок 12 – после добавления новых стоп-слов и до добавления пятого документа



Рисунок 13 – после добавления новых стоп-слов и пятого документа

Изначальный размер уникального словаря vocabulary составлял 26 слов. После добавления двух кастомных стоп-слов его размер уменьшился на 2 слова и стал равен 24. После добавления пятого документа с четырьмя новыми словами размер словаря увеличился на 4 и стал равен 28.

Длина числовых векторов изменялась аналогично. Конечная длина векторов равна 28 элементам.

3. Математическая интерпретация: Посмотрите на значения TF-IDF для новых слов в пятом документе. Почему их вес (значение) оказался выше, чем у слов, которые дублировались из прошлых текстов? Обоснуйте ответ, используя формулу IDF.

Веса новых слов из пятого документа выше весов слов, дублированных из прошлых текстов, так как IDF обратно пропорционален частоте встречаемости слова в корпусе.

Расчёт для слова «зонт» (встречается только в пятом документе):

TF = 1/7 = 0,143

IDF = ln(5/1) = 1,609

TF-IDF = 0,143 * 1,609 = 0,23

Расчёт для слова «идти» (встречается в двух документах):

TF = 1/7 = 0,143

IDF = ln(5/2) = 0,916

TF-IDF = 0,1309

Расчёт для слова «дождь» (встречается в трёх документах):

TF = 1/7 = 0,143

IDF = ln(5/3) = 0,51

TF-IDF = 0,143 * 0,51 = 0,073

Задание 3. Модификация регулярных выражений под сохранение числовых данных

1. Добавьте в один из документов корпуса точную временную или количественную метрику (например, «в 2026 году» или «3 плана»). Измените регулярное выражение в функции pipeline_preprocess. Текущий шаблон r'[^a-яё\s]' полностью стирает цифры. Модифицируйте его так, чтобы он сохранял в тексте и русские буквы, и арабские цифры.

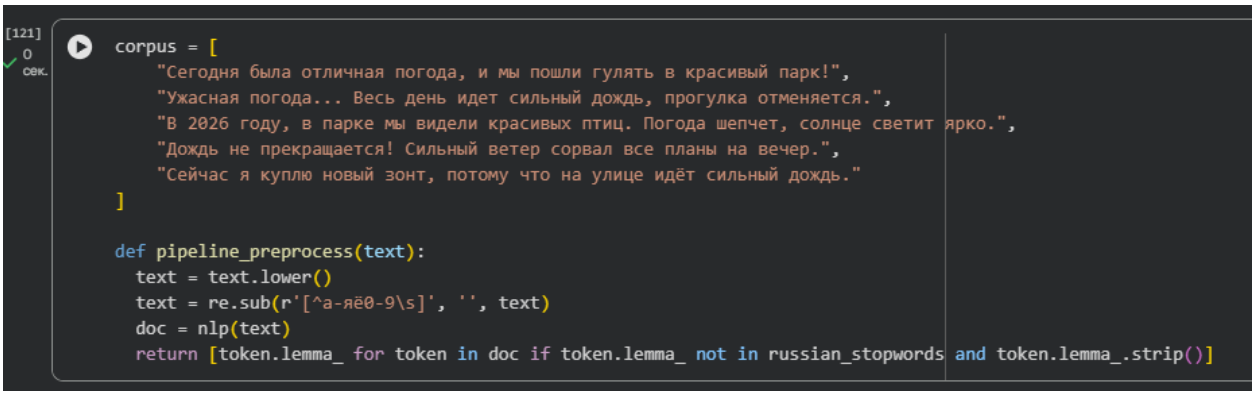


Рисунок 14 – в третий документ корпуса добавлена временная метрика «в 2026 году», шаблон модифицирован для сохранения арабских цифр

2. Прогоните обновленный текст через модель spaCy. Посмотрите, как токенизатор обработал комбинацию цифр и букв. Стала ли строка "2026" отдельным токеном? Какой начальный вид (лемму) присвоила система числительному?

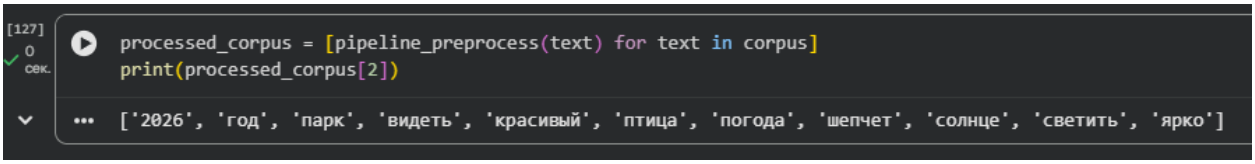


Рисунок 15 – текст с числом прогнан через модель spaCy

Строка «2026» стала отдельным токеном. Начальная форма числа совпадает с исходным значением.

3. Подумайте и напишите в выводе, в каких реальных задачах NLP сохранение цифр критически важно (например, анализ чеков, парсинг адресов), а в каких — это избыточный «шум», который нужно удалять.

Сохранение цифр критически важно в:

- анализе чеков;
- парсинге адресов и геолокации;
- анализах новостей и отчётов;
- медицинских записях;
- технических документациях.

Цифры являются избыточным «шумом» в таких задачах, как:

- фильтрация спама;
- сортировка текстов по темам;
- определение настроения текста (отзывы).

Контрольные вопросы

1. Какова роль регулярных выражений (RegEx) при предварительной очистке текстового шума?

Регулярные выражения играют ключевую роль на самом первом этапе препроцессинга, отвечая за автоматическую фильтрацию текстового шума. Они позволяют быстро очистить сырой текст от веб-тегов, системных символов, пунктуации и лишних пробелов, а также привести всё к единому регистру. Это нормализует текст и готовит его к правильному разбиению на слова.

2. К каким ошибкам в анализе может привести избыточное удаление спецсимволов и дефисов (например, в словах «e-mail» или «он-лайн»)?

Токенизатор совершит ошибки и склеит или разорвет слова вроде «e-mail» на непонятные части. В итоге компьютер посчитает их как разные новые токены, что полностью исказит весь последующий анализ.

3. Почему приведение текста к нижнему регистру (lowercase) является обязательным шагом перед токенизацией?

Приведение текста к нижнему регистру перед токенизацией обязательно, потому что для машины одинаковое слово с большой буквы и с маленькой — это два абсолютно разных слова. Перевод в lowercase стирает эту чисто визуальную разницу и объединяет их в одну лексическую единицу.

4. Как настроить регулярное выражение, чтобы оно сохраняло в тексте и буквы, и важные цифровые данные (например, даты или артикулы)?

r'[^а-яё\s]' – для сохранения букв русского алфавита, включая ё

r'[^а-яё0-9\s]' – для сохранения букв русского алфавита и арабских цифр 0-9

r'[^а-яё0-9.\s]' – для сохранения дат в формате xx.xx.xxxx

5. В каких задачах обработки текста нельзя удалять знаки препинания и смайлики (emoji)?

Удалять знаки препинания и смайлики категорически нельзя в задачах анализа тональности текста. Смайлы, восклицательные или вопросительные знаки напрямую передают сильные эмоции автора, его сарказм или разочарование. Без них алгоритм не сможет отличить нейтральное предложение от восторженного или крайне негативного отзыва.

6. В чем заключается главное различие между процессами лемматизации и стемминга?

Главное различие между лемматизацией и стеммингом заключается в подходе к обработке слова. Стемминг работает грубо и просто отсекает окончания и суффиксы, оставляя основу, которая часто даже не является реальным словом. Лемматизация — это полноценный морфологический анализ, который учитывает контекст и переводит словоформу в её каноническую словарную форму.

7. Почему для флективных языков (таких как русский) лемматизация эффективнее стемминга?

Для флективных языков лемматизация намного эффективнее стемминга. Русский язык обладает богатой системой окончаний, из-за чего у одного слова могут быть десятки разных форм. Стемминг превратит их в разные обрубки, а лемматизация за счет анализа контекста точно сведет к единой начальной форме.

8. Как алгоритм токенизации определяет границы слов при наличии сокращений (например, «т.д.», «и т.п.»)?

При обработке сокращений типа «т.д.» или «и т.п.» алгоритм токенизации ориентируется на встроенные правила и словари исключений. Он понимает, что точки внутри таких устойчивых конструкций не являются знаками конца предложения, и сохраняет сокращение как единый токен.

9. По какому принципу слова формируют список стоп-слов в библиотеках NLTK или spaCy?

В библиотеках NLTK или spaCy списки стоп-слов формируются по принципу их высокой частотности и низкой информационной ценности. Туда вносят слова, выполняющие чисто синтаксическую роль: предлоги, союзы и частицы. Они встречаются почти в каждом тексте, но не несут в себе уникального смысла

10. Что произойдет с качеством анализа тональности отзывов, если удалить частицу «не» как стоп-слово?

Без частицы «не» модель посчитает текста с противоположными значениями (например, отзыв «фильм удался» и «фильм не удался» как одно и то же.

11.Какой физический и математический смысл имеет метрика TF (Term Frequency)?

Физический смысл метрики TF заключается в оценке удельной плотности конкретного слова внутри отдельного документа. Математически это отношение количества повторений исследуемого слова к общему числу слов в этом документе.

12. Зачем при расчете метрики IDF (Inverse Document Frequency) используется операция логарифмирования?

Операция логарифмирования в формуле IDF применяется для сглаживания результатов. Без логарифма редкие слова получали бы огромный несправедливый вес.

13. Чему будет равен коэффициент TF-IDF для слова, которое встречается абсолютно во всех документах корпуса?

$IDF = \ln(N/N) = \ln(1) = 0$

$TF\text{-}IDF = TF \times 0 = 0$

Коэффициент будет равен нулю.

14. Какая часть метрики TF-IDF отвечает за локальную важность слова, а какая — за его глобальную уникальность?

TF отвечает за локальную важность — насколько слово значимо внутри конкретного документа. IDF отвечает за глобальную уникальность — насколько слово редко встречается во всём корпусе и поэтому информативно.

15. Как длина текста влияет на показатель TF и почему длинные документы могут искусственно занижать вес слов?

Длина текста сильно влияет на TF, так как в очень длинных документах знаменатель формулы (общее число слов) становится огромным. Из-за этого локальная частота нужного термина искусственно размывается и занижается. Большие тексты штрафуются алгоритмом просто за свой объем, даже если тема в них раскрыта подробно.

16. Что происходит с размером итоговой матрицы TF-IDF при добавлении в корпус новых уникальных слов?

При добавлении в корпус новых уникальных слов размер итоговой матрицы TF-IDF увеличивается по горизонтали. Каждое новое уникальное слово расширяет общий словарь (vocabulary). А так как длина вектора каждого документа строго равна размеру этого словаря, строки матрицы становятся длиннее на количество добавленных лемм.

17. Что такое «разреженность» матрицы (Sparsity) и почему в итоговой таблице TF-IDF преобладают нули?

«Разреженность» матрицы означает, что подавляющее большинство её ячеек заполнены нулями. В итоговой таблице TF-IDF нули преобладают потому, что каждый отдельный документ содержит лишь небольшую часть слов из общего словаря всего корпуса. На позициях всех остальных слов, которых нет в данном предложении, компьютер ставит нули.

18. Как ошибка определения части речи в библиотеке spaCy может повлиять на итоговый результат лемматизации?

Если библиотека spaCy ошибется в определении части речи токена, это напрямую испортит результат лемматизации. Модель перепутает омонимы — например, посчитает слово «течь» существительным, а не глаголом. Из-за этого начальной формой станет «течь» вместо инфинитива «течь» (или наоборот, глагол «стекло» превратится в «стечь» вместо существительного «стекло»).

19. В чем плюсы и минусы перехода от анализа одиночных слов к анализу словосочетаний (биграмм и триграмм)?

Плюс в том, что модель начинает улавливать контекст и устойчивые выражения, смысл которых теряется по отдельности. Минус заключается в росте словаря, из-за чего итоговая матрица становится гигантской, а вычисления требуют намного больше памяти.

20. Что произойдет, если в векторизатор TF-IDF передать новое слово, которого вообще не было в исходном обучающем словаре?

Модель проигнорирует новое слово. Векторизатор имеет строго фиксированную длину вектора под те слова, которые он видел при обучении. Для незнакомого токена в этой структуре просто нет зарезервированного индекса, поэтому на вектор документа он никак не повлияет.

Вывод

В ходе этой лабораторной работы я изучил базовые этапы предобработки текста: токенизацию, очистку с помощью регулярных выражений, удаление стоп-слов, лемматизацию с помощью spaCy и NLTK, а также векторизацию текста методом TF-IDF. Мне удалось сформировать корпус из пяти документов и запустить пайплайн-предобработку этого корпуса. С помощью функции я математически вычислил показатель TF-IDF для каждого слова в корпусе. Полученный результат наглядно продемонстрировал зависимость этого показателя от частоты слова в документе и обратной частоты документа. Затем я смог дополнить этот корпус шестым документом. Добавление документа увеличило словарь и длину векторов, а также пересчитало значение IDF каждого токена. Для сохранения числовых данных в процессе токенизации я модифицировал регулярное выражение, после чего проанализировал, как spaCy токенизирует и лемматизирует числительные (сохраняет их исходную форму).